QuickBite Manual Deployment Guide for Amazon EKS

This guide is the implementation path for deploying QuickBite manually to Amazon EKS. It covers provisioning the cluster, installing the add-ons, publishing the images, implementing the Kubernetes manifests, and validating the result.

Each implementation step contains:

- **Objective:** what the step delivers.
- **Requirements:** the specification the manifests must satisfy.
- **Commands:** the commands used to apply and inspect the result.
- **Expected result:** what a correct implementation produces.
- **Checkpoint:** the condition that must be true before you continue.

Infrastructure prerequisites (the cluster definition and controller installation) are provided in full. You write the Kubernetes manifests for QuickBite yourself, following the requirements in each step and the layout in `kubernetes/README.md`.

Contents

- Project Overview
- Deployment Requirements
- Architecture
- Prerequisites
- Implementation Steps
    - Step 1: Prepare the Workstation
    - Step 2: Validate the Application Locally
    - Step 3: Provision the Amazon EKS Cluster
    - Step 4: Install Cluster Add-ons and Controllers
    - Step 5: Build and Publish Container Images
    - Step 6: Namespace, ServiceAccounts, and RBAC
    - Step 7: ConfigMaps and Secrets
    - Step 8: Storage
    - Step 9: Data Tier StatefulSets
    - Step 10: Database Initialization Job
    - Step 11: Application Deployments and Services
    - Step 12: External Access with Gateway API
    - Step 13: NetworkPolicies
    - Step 14: Horizontal Pod Autoscaling
    - Step 15: PodDisruptionBudgets
    - Step 16: Backup CronJob
    - Step 17: End-to-End Validation
- Operations
- Cleanup
- Out of Scope

Project Overview

QuickBite consists of four application images built from this repository and two public infrastructure images:

Component	Image	Port	Kubernetes Service name
frontend	`<ECR_REGISTRY>/quickbite/frontend:<IMAGE_TAG>`	8080	`frontend`
restaurant-api	`<ECR_REGISTRY>/quickbite/restaurant-api:<IMAGE_TAG>`	3001	`restaurant-api`
orders-api	`<ECR_REGISTRY>/quickbite/orders-api:<IMAGE_TAG>`	3002	`orders-api`
notification-worker	`<ECR_REGISTRY>/quickbite/notification-worker:<IMAGE_TAG>`	none	none
PostgreSQL	`postgres:16-alpine`	5432	`postgres`
Redis	`redis:7-alpine`	6379	`redis`

The Service names are fixed requirements. `frontend/nginx.conf` proxies to `restaurant-api:3001` and `orders-api:3002`, and `orders-api` and `notification-worker` default to `redis://redis:6379`.

Deployment Requirements

The finished deployment must satisfy these requirements:

ID	Requirement
DR-01	All QuickBite resources run in the `quickbite` namespace.
DR-02	Application images are pulled from Amazon ECR using immutable tags. No workload uses the `latest` tag.
DR-03	No credential appears in a committed manifest, a ConfigMap, or a container image.
DR-04	Each workload uses a dedicated ServiceAccount with API token automounting disabled.
DR-05	Every container declares CPU and memory requests and limits.
DR-06	Every HTTP service has liveness and readiness probes. PostgreSQL and Redis have exec probes.
DR-07	PostgreSQL and Redis data persists on encrypted gp3 EBS volumes and survives Pod deletion.
DR-08	The schema and seed data are applied by a Job that can be re-run safely without duplicating data.
DR-09	The application is reachable from the internet only through the Gateway, over HTTPS, and HTTP redirects to HTTPS.
DR-10	`GET /api/orders` (administrative listing) is not reachable through the public endpoint. `POST /api/orders` remains available.
DR-11	The namespace denies all ingress and egress traffic by default. Only the flows in the traffic matrix are allowed.
DR-12	`frontend`, `restaurant-api`, and `orders-api` run at least 2 replicas spread across Availability Zones, scale on CPU through HPA, and are protected by PodDisruptionBudgets.
DR-13	A CronJob performs a daily logical backup of PostgreSQL to a dedicated PersistentVolumeClaim.
DR-14	Operations staff have read-only access to the namespace through RBAC, and they cannot read Secrets.

Architecture

Cluster Topology

- One EKS cluster (`quickbite`) in a dedicated VPC created by `eksctl` , with public and private subnets across three Availability Zones.
- One managed node group in private subnets. Outbound internet access goes through a NAT gateway.
- An internet-facing AWS Network Load Balancer, provisioned for the Gateway API data plane by the AWS Load Balancer Controller.
- EBS volumes provisioned dynamically by the Amazon EBS CSI driver.
- Network policy enforcement provided by the Amazon VPC CNI.

Namespaces

Namespace	Contents	Managed by
`quickbite`	All QuickBite workloads, configuration, storage claims, Gateway, and HTTPRoute	This project
`envoy-gateway-system`	Envoy Gateway controller and Envoy proxy Pods (Gateway data plane)	Helm
`kube-system`	CoreDNS, VPC CNI, EBS CSI driver, Metrics Server, AWS Load Balancer Controller	EKS add-ons and Helm

Network Traffic Matrix

These are the only flows the NetworkPolicies may allow:

Source	Destination	Port	Purpose
Envoy proxy Pods (`envoy-gateway-system`)	frontend	8080/TCP	Public HTTP traffic
frontend	restaurant-api	3001/TCP	Restaurant and menu API
frontend	orders-api	3002/TCP	Order API
restaurant-api	postgres	5432/TCP	Data access
orders-api	postgres	5432/TCP	Data access
orders-api	redis	6379/TCP	Enqueue notifications
notification-worker	redis	6379/TCP	Consume notifications
db-init Job	postgres	5432/TCP	Schema and seed
postgres-backup CronJob	postgres	5432/TCP	Backup
All Pods in `quickbite`	CoreDNS (`kube-system`)	53/UDP, 53/TCP	DNS resolution

Object Inventory

Kind	Name	Purpose
Namespace	`quickbite`	Workload isolation
ServiceAccount	`frontend`, `restaurant-api`, `orders-api`, `notification-worker`, `postgres`, `redis`, `db-jobs`	Per-workload identity
Role / RoleBinding	`quickbite-readonly`	Operations read-only access
ConfigMap	`quickbite-config`	Non-secret application settings
ConfigMap	`frontend-nginx`	Kubernetes-specific Nginx configuration
ConfigMap	`db-init-sql`	Contents of `database/init.sql`
Secret	`quickbite-db`	Database credentials and connection string
Secret	`quickbite-tls`	TLS certificate for the Gateway HTTPS listener
StorageClass	`gp3`	Encrypted gp3 EBS volumes
PersistentVolumeClaim	`postgres-backups`	Backup storage
StatefulSet + headless Service	`postgres`, `redis`	Data tier
Deployment + Service	`frontend`, `restaurant-api`, `orders-api`	Application tier
Deployment	`notification-worker`	Queue consumer
Job	`db-init`	Schema initialization and seed
CronJob	`postgres-backup`	Daily backups
GatewayClass, Gateway, HTTPRoute	`quickbite`	External traffic entry
NetworkPolicy	see Step 13	Traffic segmentation
HorizontalPodAutoscaler	`frontend`, `restaurant-api`, `orders-api`	CPU-based scaling

Kind	Name	Purpose
PodDisruptionBudget	`frontend`, `restaurant-api`, `orders-api`	Voluntary disruption protection

Prerequisites

Tools

Tool	Minimum version	Verify with
AWS CLI	v2	`aws --version`
eksctl	Recent release that supports your chosen Kubernetes version	`eksctl version`
kubectl	Within one minor version of the cluster	`kubectl version --client`
Helm	v3	`helm version`
Docker	With Buildx	`docker buildx version`
curl, openssl	Any current release	`curl --version`, `openssl version`
hey (optional)	Any	Load generation for HPA validation

All commands in this guide use Bash syntax. On Windows, run them from WSL 2 or Git Bash.

AWS Account

- An AWS account and IAM principal with permission to create EKS clusters, VPCs, IAM roles and policies, EC2 instances, EBS volumes, Elastic Load Balancing resources, and ECR repositories.
- Service quotas that allow at least 3 additional EC2 instances, 1 NAT gateway, 1 Network Load Balancer, and 4 EBS volumes in the target Region.
- A DNS name for the application (for example `quickbite.example.com`) if you plan to use a publicly trusted certificate. Without one, use a self-signed certificate as described in Step 12.

> **Cost notice:** This deployment creates billable resources: the EKS control plane, EC2 instances, a NAT gateway, a Network Load Balancer, EBS volumes, and ECR storage. Run Cleanup when you no longer need the environment.

Implementation Steps

Step 1: Prepare the Workstation

Objective: Set the environment variables that every later step uses.

Commands:

```
export AWS_REGION=us-east-1
export CLUSTER_NAME=quickbite
export NAMESPACE=quickbite
export AWS_ACCOUNT_ID="$(aws sts get-caller-identity --query Account --output text)"
export ECR_REGISTRY="${AWS_ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com"
export IMAGE_TAG="1.0.0"
export APP_HOST="quickbite.example.com"
```

Use a unique, immutable `IMAGE_TAG` for every build, such as a semantic version or a Git commit SHA. If you change `AWS_REGION` or `CLUSTER_NAME`, update `infrastructure/eks/cluster.yaml` to match.

Expected result: `echo $ECR_REGISTRY` prints a value in the form `123456789012.dkr.ecr.us-east-1.amazonaws.com`.

Checkpoint: `aws sts get-caller-identity` returns the intended account and principal.

Step 2: Validate the Application Locally

Objective: Confirm that the application works before you introduce Kubernetes.

Commands:

```
docker compose up --build -d
curl -s http://localhost:8080/healthz
curl -s http://localhost:8080/api/restaurants | head -c 200; echo
curl -s -X POST http://localhost:8080/api/orders \
  -H 'Content-Type: application/json' \
  -d '{"customer_name":"Local Test","items":[{"id":1,"quantity":2}]}'
docker compose logs notification-worker | tail -n 5
docker compose down -v
```

Expected result:

- `/healthz` returns `ok`.
- `/api/restaurants` returns JSON starting with `[{"id":1,"name":"Burger House"`.
- The order returns HTTP 201 with `"total":"27.00"`.
- The worker log contains `SIMULATED NOTIFICATION:`.

Checkpoint: The full order flow works locally. Any failure here is an application or build problem, not a Kubernetes problem.

Step 3: Provision the Amazon EKS Cluster

Objective: Create the cluster, VPC, private managed node group, and core add-ons.

The cluster definition is in `infrastructure/eks/cluster.yaml`. It does the following:

- Creates a new VPC with a single NAT gateway.
- Places worker nodes in private subnets.
- Enables the IAM OIDC provider for IAM Roles for Service Accounts (IRSA).
- Installs the `vpc-cni` add-on with network policy enforcement enabled.
- Installs the `aws-ebs-csi-driver` add-on with an IRSA role.
- Enables control plane logging for the API server, audit, and authenticator logs.

Review the file and set `metadata.region` and `metadata.version`. Choose a Kubernetes version that is in EKS standard support.

Commands:

```
eksctl create cluster -f infrastructure/eks/cluster.yaml
aws eks update-kubeconfig --region "$AWS_REGION" --name "$CLUSTER_NAME"
kubectl get nodes -o wide
eksctl get addons --cluster "$CLUSTER_NAME" --region "$AWS_REGION"
```

Provisioning takes about 15 to 25 minutes.

Expected result:

- 3 nodes in `Ready` state, with internal IPs only (the `EXTERNAL-IP` column shows `<none>`).
- The add-ons `vpc-cni`, `coredns`, `kube-proxy`, and `aws-ebs-csi-driver` are `ACTIVE`.

Checkpoint:

```
kubectl get pods -n kube-system
kubectl get nodes -L topology.kubernetes.io/zone
```

All `kube-system` Pods are `Running`, and the nodes are spread across more than one Availability Zone.

Step 4: Install Cluster Add-ons and Controllers

Objective: Install the components that the QuickBite manifests depend on: Metrics Server (HPA), the AWS Load Balancer Controller (NLB provisioning), and a Gateway API implementation.

4.1 Metrics Server

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-
server/releases/latest/download/components.yaml
kubectl -n kube-system rollout status deployment/metrics-server --timeout=180s
kubectl top nodes
```

Expected result: `kubectl top nodes` returns CPU and memory figures for each node. The first metrics can take up to 60 seconds to appear.

4.2 AWS Load Balancer Controller

```
curl -fsSL -o iam_policy.json \
  https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-
controller/main/docs/install/iam_policy.json

aws iam create-policy \
  --policy-name AWSLoadBalancerControllerIAMPolicy \
  --policy-document file://iam_policy.json

eksctl create iamserviceaccount \
  --cluster "$CLUSTER_NAME" --region "$AWS_REGION" \
  --namespace kube-system \
  --name aws-load-balancer-controller \
  --attach-policy-arn
"arn:aws:iam::${AWS_ACCOUNT_ID}:policy/AWSLoadBalancerControllerIAMPolicy" \
  --override-existing-serviceaccounts \
  --approve

export VPC_ID="$(aws eks describe-cluster --name "$CLUSTER_NAME" --region "$AWS_REGION" \
  --query 'cluster.resourcesVpcConfig.vpcId' --output text)"

helm repo add eks https://aws.github.io/eks-charts
helm repo update
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  --namespace kube-system \
  --set clusterName="$CLUSTER_NAME" \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller \
  --set region="$AWS_REGION" \
  --set vpcId="$VPC_ID"

kubectl -n kube-system rollout status deployment/aws-load-balancer-controller --timeout=180s
```

If the policy already exists in the account, skip `aws iam create-policy`.

Expected result: The `aws-load-balancer-controller` Deployment reports 2/2 replicas available.

4.3 Gateway API Implementation

This guide uses **Envoy Gateway** as the reference Gateway API implementation. Its Helm chart installs the standard Gateway API CRDs. If your organization standardizes on a different conformant implementation (for example the AWS Load Balancer Controller's Gateway API support, NGINX Gateway Fabric, or Istio), install it according to its documentation. Then adjust the `GatewayClass`, the proxy namespace in the NetworkPolicies, and the load balancer settings to match.

Check the Envoy Gateway releases page for the current stable version, then install it:

```
export ENVOY_GATEWAY_VERSION=v1.5.0   # replace with the current stable release

helm install eg oci://docker.io/envoyproxy/gateway-helm \
  --version "$ENVOY_GATEWAY_VERSION" \
  --namespace envoy-gateway-system --create-namespace

kubectl -n envoy-gateway-system wait --for=condition=Available deployment/envoy-gateway --
timeout=300s
kubectl get crd gateways.gateway.networking.k8s.io httproutes.gateway.networking.k8s.io
```

If you use an implementation that does not bundle the CRDs, install the standard channel CRDs first:

```
kubectl apply --server-side -f \
  https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.3.0/standard-install.yaml
```

Expected result: The `envoy-gateway` Deployment is `Available`, and the Gateway API CRDs exist.

4.4 Default StorageClass Check

```
kubectl get storageclass
```

Recent EKS versions do not mark any StorageClass as the default. QuickBite defines its own `gp3` StorageClass in Step 8 and references it explicitly, so it does not depend on a cluster default.

Checkpoint: Metrics Server returns node metrics, the AWS Load Balancer Controller is running, and `kubectl api-resources | grep gateway.networking.k8s.io` lists `gatewayclasses`, `gateways`, and `httproutes`.

Step 5: Build and Publish Container Images

Objective: Publish the four application images to private ECR repositories.

Commands:

```
for repo in frontend restaurant-api orders-api notification-worker; do
  aws ecr create-repository \
    --repository-name "quickbite/${repo}" \
    --image-tag-mutability IMMUTABLE \
    --image-scanning-configuration scanOnPush=true \
    --encryption-configuration encryptionType=AES256 \
    --region "$AWS_REGION"
done

aws ecr get-login-password --region "$AWS_REGION" \
  | docker login --username AWS --password-stdin "$ECR_REGISTRY"

docker buildx build --platform linux/amd64 \
  -t "${ECR_REGISTRY}/quickbite/frontend:${IMAGE_TAG}" --push ./frontend

for svc in restaurant-api orders-api notification-worker; do
  docker buildx build --platform linux/amd64 \
    -t "${ECR_REGISTRY}/quickbite/${svc}:${IMAGE_TAG}" --push "./services/${svc}"
done
```

The `--platform linux/amd64` flag matches the `t3.large` nodes in the cluster definition. It is mandatory when you build on Apple Silicon or another ARM host. If you change the node group to Graviton instances, build for `linux/arm64` instead, or publish multi-architecture images.

Expected result:

```
for repo in frontend restaurant-api orders-api notification-worker; do
  aws ecr describe-images --repository-name "quickbite/${repo}" --region "$AWS_REGION" \
    --query 'imageDetails[].imageTags' --output text
done
```

Each repository lists the tag `1.0.0` (or your `IMAGE_TAG`).

```
aws ecr describe-image-scan-findings --repository-name quickbite/orders-api \
  --image-id imageTag="$IMAGE_TAG" --region "$AWS_REGION" \
  --query 'imageScanFindings.findingSeverityCounts'
```

Review any `CRITICAL` or `HIGH` findings before you deploy.

Checkpoint: All four images exist in ECR with the same immutable tag. The node IAM role created by `eksctl` already includes `AmazonEC2ContainerRegistryReadOnly`, so you do not need `imagePullSecrets`.

> **Public base images:** `postgres:16-alpine` and `redis:7-alpine` are pulled from Docker Hub, which applies pull rate limits. For production, mirror them into ECR or configure an ECR pull through cache rule, and reference the ECR copies.

Step 6: Namespace, ServiceAccounts, and RBAC

Objective: Create the isolation boundary, workload identities, and read-only operations access.

Manifest location: `kubernetes/00-namespace/`, `kubernetes/01-rbac/`

Requirements:

1. Create a Namespace named `quickbite` with these labels:
   - `app.kubernetes.io/part-of: quickbite`
   - Pod Security Admission labels `pod-security.kubernetes.io/warn: restricted` and `pod-security.kubernetes.io/audit: restricted`. Do not enforce `restricted`, because the stock `nginx`, `postgres`, and `redis` images start as root.
2. Create one ServiceAccount per workload: `frontend`, `restaurant-api`, `orders-api`, `notification-worker`, `postgres`, `redis`, and `db-jobs` (shared by the Job and the CronJob).
3. Set `automountServiceAccountToken: false` on every ServiceAccount. No QuickBite component calls the Kubernetes API.
4. Create a Role `quickbite-readonly` in the `quickbite` namespace that:
   - Grants `get`, `list`, and `watch` on `pods`, `pods/log`, `services`, `endpoints`, `configmaps`, `persistentvolumeclaims`, `events`, `deployments`, `replicasets`, `statefulsets`, `jobs`, `cronjobs`, `horizontalpodautoscalers`, `poddisruptionbudgets`, `networkpolicies`, `gateways`, and `httproutes`.
   - Does **not** grant any access to `secrets`, and does not grant `pods/exec` or `pods/portforward`.
5. Create a RoleBinding that binds `quickbite-readonly` to the Kubernetes group `quickbite-readonly`.

Commands:

```
kubectl apply -f kubernetes/00-namespace/
kubectl apply -f kubernetes/01-rbac/
```

To give an IAM role (for example an operations team role) access to the cluster through that group, create an EKS access entry:

```
export OPS_ROLE_ARN="arn:aws:iam::${AWS_ACCOUNT_ID}:role/<operations-role-name>"
aws eks create-access-entry \
  --cluster-name "$CLUSTER_NAME" --region "$AWS_REGION" \
  --principal-arn "$OPS_ROLE_ARN" \
  --kubernetes-groups quickbite-readonly
```

Expected result:

```
kubectl get namespace quickbite --show-labels
kubectl -n quickbite get serviceaccounts
kubectl -n quickbite auth can-i list pods      --as=ops-user --as-group=quickbite-readonly   #
yes
kubectl -n quickbite auth can-i get pods/log   --as=ops-user --as-group=quickbite-readonly   #
yes
kubectl -n quickbite auth can-i get secrets    --as=ops-user --as-group=quickbite-readonly   #
no
kubectl -n quickbite auth can-i create pods/exec --as=ops-user --as-group=quickbite-readonly #
no
kubectl -n quickbite auth can-i delete pods    --as=ops-user --as-group=quickbite-readonly   #
no
kubectl auth can-i list pods -n kube-system    --as=ops-user --as-group=quickbite-readonly   #
no
```

Checkpoint: The namespace exists, all 7 ServiceAccounts exist, and every `auth can-i` result matches the expected value.

Step 7: ConfigMaps and Secrets

Objective: Externalize configuration and credentials, and close the public exposure of the order listing endpoint.

Manifest location: `kubernetes/02-config/`

7.1 Application ConfigMap

Requirements: Create a ConfigMap `quickbite-config` with:

Key	Value
`REDIS_URL`	`redis://redis:6379`
`ALLOWED_ORIGIN`	`https://<APP_HOST>` (your application URL)

7.2 Nginx ConfigMap

The image's built-in `frontend/nginx.conf` proxies every method on `/api/orders`, which exposes the administrative order listing publicly (DR-10). Override the configuration without changing the image.

Requirements: Create a ConfigMap `frontend-nginx` with the key `default.conf`. It must:

1. Listen on port 8080 and serve `/usr/share/nginx/html`, with the same `try_files` fallback as the source file.
2. Proxy `/api/restaurants` to `http://restaurant-api.quickbite.svc.cluster.local:3001`.
3. Proxy `/api/orders` to `http://orders-api.quickbite.svc.cluster.local:3002`, accepting only `POST`. Any other method must be rejected (for example with `limit_except POST { deny all; }`, which returns 403).
4. Keep `location /healthz { return 200 'ok'; }`.

In Step 11, mount it into the frontend container at `/etc/nginx/conf.d/`, replacing the image's `default.conf`. Nginx reads its configuration only at startup, so you must restart the frontend Deployment after any change to this ConfigMap.

7.3 Database Initialization ConfigMap

Generate the ConfigMap directly from the repository file so that the SQL has a single source of truth:

```
kubectl create configmap db-init-sql \
  --namespace quickbite \
  --from-file=init.sql=database/init.sql \
  --dry-run=client -o yaml > kubernetes/02-config/db-init-sql-configmap.yaml
```

7.4 Database Secret

Requirements:

- Create the Secret `quickbite-db` with the keys `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`, and `DATABASE_URL`.
- Do not commit the Secret to source control. Create it imperatively, or use an external secrets integration (for example AWS Secrets Manager with the External Secrets Operator).
- Use a password without URL-reserved characters, or URL-encode it inside `DATABASE_URL`. The command below uses hexadecimal output to avoid encoding problems.

```
export POSTGRES_PASSWORD="$(openssl rand -hex 24)"
kubectl -n quickbite create secret generic quickbite-db \
  --from-literal=POSTGRES_USER=quickbite \
  --from-literal=POSTGRES_PASSWORD="$POSTGRES_PASSWORD" \
  --from-literal=POSTGRES_DB=quickbite \
  --from-
literal=DATABASE_URL="postgres://quickbite:${POSTGRES_PASSWORD}@postgres:5432/quickbite"
```

Store the password in your organization's secret manager. PostgreSQL applies `POSTGRES_PASSWORD` only when it first initializes an empty volume. Changing the Secret afterwards does not change the database password (see Troubleshooting).

The TLS Secret `quickbite-tls` is created in Step 12.

Commands:

```
kubectl apply -f kubernetes/02-config/
kubectl -n quickbite get configmaps,secrets
```

Expected result: The ConfigMaps `quickbite-config`, `frontend-nginx`, and `db-init-sql` and the Secret `quickbite-db` exist. `kubectl -n quickbite get secret quickbite-db -o jsonpath='{.data}' | grep -o '"[A-Z_]*"'` lists the 4 keys.

Checkpoint: `git status` shows no file containing the database password.

Step 8: Storage

Objective: Provide encrypted, zone-aware block storage for the data tier and backups.

Manifest location: `kubernetes/03-storage/`

Requirements:

1. Create a StorageClass `gp3`:
   - `provisioner: ebs.csi.aws.com`
   - parameters `type: gp3` and `encrypted: "true"`
   - `volumeBindingMode: WaitForFirstConsumer`, so the volume is created in the same Availability Zone as the Pod that uses it
   - `allowVolumeExpansion: true`
   - `reclaimPolicy: Delete` (or `Retain` if your data retention policy requires it; document the choice)
2. Create a standalone PersistentVolumeClaim `postgres-backups`: `storageClassName: gp3`, `ReadWriteOnce`, `10Gi`.

Commands:

```
kubectl apply -f kubernetes/03-storage/
kubectl get storageclass gp3
kubectl -n quickbite get pvc postgres-backups
```

Expected result: The StorageClass exists. The PVC `postgres-backups` stays `Pending` with the event `waiting for first consumer to be created before binding`. This is correct. It binds when the backup CronJob first runs in Step 16.

Checkpoint: `kubectl describe storageclass gp3` shows `VolumeBindingMode: WaitForFirstConsumer`.

Step 9: Data Tier StatefulSets

Objective: Run PostgreSQL and Redis with stable identities and persistent volumes.

Manifest location: `kubernetes/04-data/`

9.1 PostgreSQL

Requirements:

1. A headless Service `postgres` (`clusterIP: None`) exposing port 5432, named `postgres`.
2. A StatefulSet `postgres`:
   - `serviceName: postgres`, `replicas: 1`, `serviceAccountName: postgres`
   - image `postgres:16-alpine`
   - `POSTGRES_USER`, `POSTGRES_PASSWORD`, and `POSTGRES_DB` from the Secret `quickbite-db`
   - `PGDATA=/var/lib/postgresql/data/pgdata`, required because an EBS ext4 file system contains `lost+found` at its root
   - a `volumeClaimTemplates` entry `data` (`gp3`, `ReadWriteOnce`, `20Gi`) mounted at `/var/lib/postgresql/data`
   - readiness and liveness exec probes running `pg_isready -U quickbite -d quickbite`
   - requests `250m` CPU / `512Mi` memory, limits `1` CPU / `1Gi` memory
3. Do **not** mount `init.sql` into `/docker-entrypoint-initdb.d`. Schema creation is the responsibility of the Job in Step 10. Using both methods duplicates the seed data.

9.2 Redis

Requirements:

1. A headless Service `redis` exposing port 6379.
2. A StatefulSet `redis`:
   - `serviceName: redis`, `replicas: 1`, `serviceAccountName: redis`
   - image `redis:7-alpine`, with args `["redis-server", "--appendonly", "yes"]` (matching `compose.yaml`)
   - a volume claim template `data` (`gp3`, `5Gi`) mounted at `/data`
   - readiness and liveness exec probes running `redis-cli ping`
   - requests `100m` / `128Mi`, limits `500m` / `256Mi`

Commands:

```
kubectl apply -f kubernetes/04-data/
kubectl -n quickbite rollout status statefulset/postgres --timeout=300s
kubectl -n quickbite rollout status statefulset/redis --timeout=300s
kubectl -n quickbite get pods,pvc -l app.kubernetes.io/component=database
```

Expected result:

- `postgres-0` and `redis-0` are `Running` and `1/1` ready.
- The PVCs `data-postgres-0` and `data-redis-0` are `Bound`.

```
kubectl -n quickbite exec postgres-0 -- pg_isready -U quickbite -d quickbite
# /var/run/postgresql:5432 - accepting connections
kubectl -n quickbite exec redis-0 -- redis-cli ping
# PONG
```

Checkpoint: Delete `postgres-0` (`kubectl -n quickbite delete pod postgres-0`). The Pod is recreated with the same name, reattaches the same PVC, and becomes ready again.

Step 10: Database Initialization Job

Objective: Apply the schema and seed data exactly once, in a repeatable and auditable way.

Manifest location: `kubernetes/05-jobs/db-init-job.yaml`

Requirements:

1. A Job `db-init` using `serviceAccountName: db-jobs` and image `postgres:16-alpine`.
2. Mount the ConfigMap `db-init-sql` at `/sql`.
3. Connect to host `postgres`, database `quickbite`, user `quickbite`. Supply the password through the `PGPASSWORD` environment variable from the Secret `quickbite-db`, key `POSTGRES_PASSWORD`.
4. **Idempotency (DR-08):** `init.sql` uses `CREATE TABLE IF NOT EXISTS`, but its `INSERT` statements are not idempotent. The Job must apply the file only when the `restaurants` table is absent or empty, and otherwise exit successfully without changes.
5. Run the file in a single transaction that stops on the first error: `psql -v ON_ERROR_STOP=1 --single-transaction -f /sql/init.sql`.
6. Wait for PostgreSQL to accept connections before running (for example with a `pg_isready` loop), so the Job does not use up its retries while the database starts.
7. Set `backoffLimit: 4`, `restartPolicy: OnFailure` or `Never`, and `ttlSecondsAfterFinished: 86400`.

Commands:

```
kubectl apply -f kubernetes/05-jobs/db-init-job.yaml
kubectl -n quickbite wait --for=condition=complete job/db-init --timeout=180s
kubectl -n quickbite logs job/db-init
kubectl -n quickbite exec postgres-0 -- psql -U quickbite -d quickbite -tAc \
  "SELECT (SELECT count(*) FROM restaurants), (SELECT count(*) FROM menu_items), (SELECT
count(*) FROM orders);"
```

Expected result: The Job reaches `Complete`, and the query returns `6|7|0`.

Checkpoint (idempotency):

```
kubectl -n quickbite delete job db-init
kubectl apply -f kubernetes/05-jobs/db-init-job.yaml
kubectl -n quickbite wait --for=condition=complete job/db-init --timeout=180s
kubectl -n quickbite exec postgres-0 -- psql -U quickbite -d quickbite -tAc "SELECT count(*)
FROM restaurants;"
```

The count is still `6`.

> If the NetworkPolicies from Step 13 are already in place when you re-run the Job, the policy that allows `db-init` to reach `postgres` must also exist.

Step 11: Application Deployments and Services

Objective: Run the stateless application tier with health checks, resource controls, and hardened security contexts.

Manifest location: `kubernetes/06-apps/`

Common requirements for all four Deployments:

Setting	Requirement
Labels	`app.kubernetes.io/name: <component>`, `app.kubernetes.io/part-of: quickbite`, `app.kubernetes.io/component: frontend`, `api`, or `worker`
Image	`<ECR_REGISTRY>/quickbite/<component>:<IMAGE_TAG>`; never `latest`
`serviceAccountName`	The matching ServiceAccount from Step 6
Rolling update	`maxUnavailable: 0`, `maxSurge: 1`
Spreading	`topologySpreadConstraints` on `topology.kubernetes.io/zone`, `maxSkew: 1`, `whenUnsatisfiable: ScheduleAnyway`
Container security	`allowPrivilegeEscalation: false`, `capabilities.drop: ["ALL"]`, `seccompProfile.type: RuntimeDefault`

Per-component requirements:

Component	Replicas	Port	Environment	Probes	Requests / Limits
frontend	2	8080	none	liveness and readiness `GET /healthz` on 8080	50m / 64Mi, 250m / 128Mi
restaurant-api	2	3001	`DATABASE_URL` from Secret `quickbite-db`; `ALLOWED_ORIGIN` from ConfigMap `quickbite-config`	liveness `GET /healthz`, readiness `GET /readyz` on 3001	100m / 128Mi, 500m / 256Mi
orders-api	2	3002	`DATABASE_URL` from Secret; `REDIS_URL` and `ALLOWED_ORIGIN` from ConfigMap	liveness `GET /healthz`, readiness `GET /readyz` on 3002	100m / 128Mi, 500m / 256Mi
notification-worker	1	none	`REDIS_URL` from ConfigMap	none (no HTTP listener; the process exits if its initial Redis connection fails)	50m / 64Mi, 250m / 128Mi

Image-specific security requirements:

- **Node.js images** (`restaurant-api`, `orders-api`, `notification-worker`): The Dockerfiles set `USER node`, which is a user name, not a UID. Set `runAsNonRoot: true` **and** `runAsUser: 1000` explicitly. Without the numeric UID the kubelet cannot verify the user, and the Pod fails with `CreateContainerConfigError`. If you enable `readOnlyRootFilesystem: true`, mount an `emptyDir` at `/home/node/.npm`, because `npm start` writes log files there.
- **frontend** (`nginx:1.27-alpine`): The stock Nginx image starts its master process as root, so `runAsNonRoot` cannot be enforced. Mount the ConfigMap `frontend-nginx` as a volume at `/etc/nginx/conf.d`. Record the root requirement as a known exception (see Scope and Limitations).

Services: Create ClusterIP Services `frontend` (8080), `restaurant-api` (3001), and `orders-api` (3002). Name each port (for example `http`) and set `targetPort` to the container port. The worker has no Service.

Ordering: Nginx resolves its upstream host names at startup. Apply the Services before (or together with) the frontend Deployment. If the frontend starts before `restaurant-api` or `orders-api` Services exist, it fails with `host not found in upstream`.

Commands:

```
kubectl apply -f kubernetes/06-apps/
for d in restaurant-api orders-api notification-worker frontend; do
  kubectl -n quickbite rollout status deployment/$d --timeout=300s
done
kubectl -n quickbite get deploy,svc,endpoints
```

Expected result:

- All Deployments report all replicas ready. `notification-worker` is `1/1`.
- The Endpoints for `frontend`, `restaurant-api`, and `orders-api` each list 2 Pod IPs.
- `kubectl -n quickbite logs deploy/notification-worker` contains `QuickBite notification worker started`.

Checkpoint (in-cluster functional test through port-forward):

```
kubectl -n quickbite port-forward svc/frontend 8080:8080 &
PF_PID=$!
sleep 3
curl -s http://localhost:8080/healthz                                    # ok
curl -s http://localhost:8080/api/restaurants | head -c 120; echo        # JSON array
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/api/orders  # 403
curl -s -X POST http://localhost:8080/api/orders -H 'Content-Type: application/json' \
  -d '{"customer_name":"Cluster Test","items":[{"id":2,"quantity":1}]}'  # HTTP 201 JSON
kill $PF_PID
kubectl -n quickbite logs deploy/notification-worker --tail=5
```

The worker log shows `SIMULATED NOTIFICATION: {"order_id":1,"customer_name":"Cluster Test"}`.

Step 12: External Access with Gateway API

Objective: Expose the storefront to the internet through Gateway API over HTTPS.

Manifest location: `kubernetes/07-gateway/`

12.1 TLS Certificate

For a publicly trusted certificate, issue one for `APP_HOST` with your organization's certificate authority or with cert-manager, and store it as the TLS Secret `quickbite-tls` in the `quickbite` namespace.

For a non-production validation environment, create a self-signed certificate:

```
openssl req -x509 -nodes -newkey rsa:2048 -days 30 \
  -keyout tls.key -out tls.crt -subj "/CN=${APP_HOST}" \
  -addext "subjectAltName=DNS:${APP_HOST}"
kubectl -n quickbite create secret tls quickbite-tls --cert=tls.crt --key=tls.key
rm -f tls.key tls.crt
```

12.2 Gateway Resources

Requirements:

1. **EnvoyProxy** (`gateway.envoyproxy.io/v1alpha1`, in `envoy-gateway-system`) that makes the Envoy data plane Service an internet-facing NLB through the AWS Load Balancer Controller. Add these annotations to `spec.provider.kubernetes.envoyService.annotations`:
    - `service.beta.kubernetes.io/aws-load-balancer-type: external`
    - `service.beta.kubernetes.io/aws-load-balancer-nlb-target-type: ip`
    - `service.beta.kubernetes.io/aws-load-balancer-scheme: internet-facing`

2. **GatewayClass** `quickbite`, with `controllerName: gateway.envoyproxy.io/gatewayclass-controller` and a `parametersRef` to the EnvoyProxy resource.
3. **Gateway** `quickbite` in the `quickbite` namespace, with two listeners:
   - `http`: port 80, protocol `HTTP`, hostname `<APP_HOST>`
   - `https`: port 443, protocol `HTTPS`, hostname `<APP_HOST>`, `tls.mode: Terminate`, `certificateRefs` pointing to `quickbite-tls`
   - `allowedRoutes.namespaces.from: Same` on both listeners
4. **HTTPRoute** `quickbite-redirect` attached to the `http` listener (`sectionName: http`) with a `RequestRedirect` filter (`scheme: https`, `statusCode: 301`).
5. **HTTPRoute** `quickbite` attached to the `https` listener (`sectionName: https`), hostname `<APP_HOST>`, with one rule: `PathPrefix: /` to backend Service `frontend`, port 8080.

All public traffic must go through the frontend. Do not route directly to `restaurant-api` or `orders-api`.

Commands:

```
kubectl apply -f kubernetes/07-gateway/
kubectl get gatewayclass quickbite
kubectl -n quickbite wait --for=condition=Programmed gateway/quickbite --timeout=600s
kubectl -n quickbite get gateway quickbite
kubectl -n quickbite describe httproute quickbite | sed -n '/Status:/,$p'
export LB_HOST="$(kubectl -n quickbite get gateway quickbite -o
jsonpath='{.status.addresses[0].value}')"
echo "$LB_HOST"
```

Expected result:

- The GatewayClass `quickbite` shows `ACCEPTED: True`.
- The Gateway shows `PROGRAMMED: True` and an address ending in `.elb.<region>.amazonaws.com`.
- Both HTTPRoutes report `Accepted: True` and `ResolvedRefs: True`.

Wait 2 to 3 minutes for the NLB targets to become healthy and for DNS to propagate.

Checkpoint: If DNS for `APP_HOST` does not point to the NLB yet, use `--connect-to` to send requests to the load balancer while keeping the correct SNI and Host header:

```
curl -sk --connect-to "${APP_HOST}:443:${LB_HOST}:443" "https://${APP_HOST}/healthz"
# ok
curl -s -o /dev/null -w '%{http_code} %{redirect_url}\n' \
  --connect-to "${APP_HOST}:80:${LB_HOST}:80" "http://${APP_HOST}/"
# 301 https://quickbite.example.com/
```

Once DNS is in place, create a Route 53 alias (or CNAME) record from `APP_HOST` to `LB_HOST`. Remove `-k` when you use a trusted certificate.

Step 13: NetworkPolicies

Objective: Enforce least-privilege network access inside the namespace (DR-11).

Manifest location: `kubernetes/08-network-policies/`

Requirements:

1. `default-deny-all`: selects all Pods, `policyTypes: [Ingress, Egress]`, with no rules.
2. `allow-dns-egress`: allows egress from all Pods to the `kube-system` namespace (`kubernetes.io/metadata.name: kube-system`) with label `k8s-app: kube-dns`, on 53/UDP and 53/TCP.
3. One policy per flow in the Network Traffic Matrix. Each flow needs **both** an egress rule on the source and an ingress rule on the destination.
4. Ingress to `frontend` from the Envoy proxy Pods must use a `namespaceSelector` on `kubernetes.io/metadata.name: envoy-gateway-system`. Do not use `0.0.0.0/0`.
5. Select Pods by the `app.kubernetes.io/name` label only. Add the same label to the Job and CronJob Pod templates (`db-init`, `postgres-backup`).

Commands:

```
kubectl apply -f kubernetes/08-network-policies/
kubectl -n quickbite get networkpolicy
```

Expected result (allowed flows): The public checks from Step 12 still pass, a test order still produces a worker notification, and all Pods remain ready.

Expected result (denied flows):

```
# A Pod without an allowed label cannot reach the database
kubectl -n quickbite run np-test --rm -it --restart=Never --image=postgres:16-alpine -- \
  sh -c 'pg_isready -h postgres -p 5432 -t 5; echo exit=$?'
# expected: "postgres:5432 - no response" and exit=2

# Positive control: the frontend can reach restaurant-api
kubectl -n quickbite exec deploy/frontend -- wget -q -T 3 -O - http://restaurant-
api:3001/healthz
# expected: {"status":"ok"}

# The notification worker cannot reach restaurant-api (no such flow in the matrix)
kubectl -n quickbite exec deploy/notification-worker -- \
  sh -c 'wget -q -T 3 -O - http://restaurant-api:3001/healthz; echo exit=$?'
# expected: a timeout message and a non-zero exit code
```

Checkpoint: All allowed flows work, and both denied flows fail. If everything fails right after you apply the default deny policy, check the DNS egress policy first.

Step 14: Horizontal Pod Autoscaling

Objective: Scale the customer-facing tier automatically on CPU utilization (DR-12).

Manifest location: `kubernetes/09-autoscaling/`

Requirements: Create `autoscaling/v2` HorizontalPodAutoscalers:

Target	minReplicas	maxReplicas	Metric
frontend	2	6	CPU averageUtilization 70
restaurant-api	2	6	CPU averageUtilization 70
orders-api	2	6	CPU averageUtilization 70

Add a scale-down stabilization window of 300 seconds under `behavior.scaleDown.stabilizationWindowSeconds`. HPA calculates utilization from CPU **requests**, so the requests from Step 11 must be present.

Commands:

```
kubectl apply -f kubernetes/09-autoscaling/
kubectl -n quickbite get hpa
```

Expected result: After about 1 minute, the `TARGETS` column shows real percentages (for example `cpu: 3%/70%`), not `<unknown>`.

Checkpoint (load test):

```
hey -z 3m -c 50 "https://${APP_HOST}/api/restaurants"     # trusted DNS and certificate
# or, without DNS, a curl loop:
# for i in $(seq 1 20); do (while true; do curl -sk --connect-to
"${APP_HOST}:443:${LB_HOST}:443" \
#   "https://${APP_HOST}/api/restaurants" >/dev/null; done) & done; sleep 180; kill $(jobs -p)

kubectl -n quickbite get hpa restaurant-api -w
```

During the load test the `restaurant-api` replicas increase above 2. They return to 2 a few minutes after the load stops.

Step 15: PodDisruptionBudgets

Objective: Keep the service available during voluntary disruptions such as node drains and upgrades (DR-12).

Manifest location: `kubernetes/10-availability/`

Requirements:

1. Create PodDisruptionBudgets for `frontend`, `restaurant-api`, and `orders-api` with `minAvailable: 1`, selecting on `app.kubernetes.io/name`.
2. Do not create a PDB for the single-replica `postgres` or `redis` StatefulSets with `minAvailable: 1`, because it would block every node drain indefinitely. Document how you plan the maintenance window for the data tier.

Commands:

```
kubectl apply -f kubernetes/10-availability/
kubectl -n quickbite get pdb
```

Expected result: Each PDB shows `ALLOWED DISRUPTIONS: 1` when its Deployment has 2 ready replicas.

Checkpoint (drain test):

```
NODE="$(kubectl -n quickbite get pod -l app.kubernetes.io/name=orders-api -o
jsonpath='{.items[0].spec.nodeName}')"
kubectl drain "$NODE" --ignore-daemonsets --delete-emptydir-data --timeout=300s
kubectl -n quickbite get pods -o wide
kubectl uncordon "$NODE"
```

While the node drains, keep requesting `/healthz` and `/api/restaurants` through the Gateway. The requests keep succeeding (brief data tier unavailability is expected if `postgres-0` or `redis-0` ran on the drained node). The EBS volume is zonal, so a rescheduled `postgres-0` can only run on a node in the same Availability Zone.

Step 16: Backup CronJob

Objective: Take a daily logical backup of the order database (DR-13).

Manifest location: `kubernetes/05-jobs/postgres-backup-cronjob.yaml`

Requirements:

1. A CronJob `postgres-backup`, `schedule: "0 2 * * *"`, with `timeZone: "Etc/UTC"`.
2. `concurrencyPolicy: Forbid`, `successfulJobsHistoryLimit: 3`, `failedJobsHistoryLimit: 3`, and `startingDeadlineSeconds: 600`.
3. Image `postgres:16-alpine`, `serviceAccountName: db-jobs`, and Pod label `app.kubernetes.io/name: postgres-backup`.
4. Mount the PVC `postgres-backups` at `/backups`.
5. Run `pg_dump -h postgres -U quickbite -d quickbite -Fc` to a timestamped file such as `/backups/quickbite-YYYYMMDDTHHMMSSZ.dump`. Take `PGPASSWORD` from the Secret `quickbite-db`.
6. Delete dump files older than 7 days after a successful dump.
7. Fail the Job (non-zero exit) if `pg_dump` fails.

Commands:

```
kubectl apply -f kubernetes/05-jobs/postgres-backup-cronjob.yaml
kubectl -n quickbite create job --from=cronjob/postgres-backup postgres-backup-manual
kubectl -n quickbite wait --for=condition=complete job/postgres-backup-manual --timeout=300s
kubectl -n quickbite logs job/postgres-backup-manual
kubectl -n quickbite get pvc postgres-backups
```

Expected result: The manual Job completes, and the PVC `postgres-backups` is now `Bound`.

Checkpoint: Confirm that the dump file exists and is readable:

```
kubectl -n quickbite get cronjob postgres-backup
```

Then run a temporary Pod that mounts `postgres-backups` (with label `app.kubernetes.io/name: postgres-backup` so the NetworkPolicies allow it) and run `ls -l /backups` and `pg_restore --list /backups/<file>`. The listing shows the `restaurants`, `menu_items`, and `orders` tables.

Step 17: End-to-End Validation

Objective: Prove that the complete platform meets the deployment requirements.

Work through validation-checklist.md in full and record the results. Minimum smoke test:

```
BASE="https://${APP_HOST}"
CURL="curl -sk --connect-to ${APP_HOST}:443:${LB_HOST}:443"

$CURL "$BASE/healthz"                                        # ok
$CURL "$BASE/api/restaurants" | grep -o '"name"' | wc -l     # 6
$CURL "$BASE/api/restaurants/3/menu"                         # Creamy Alfredo
$CURL -o /dev/null -w '%{http_code}\n' "$BASE/api/orders"    # 403
$CURL -X POST "$BASE/api/orders" -H 'Content-Type: application/json' \
  -d '{"customer_name":"Validation User","items":[{"id":1,"quantity":2},
{"id":2,"quantity":1}]}'
# HTTP 201, "total":"31.50", "status":"confirmed"
kubectl -n quickbite logs deploy/notification-worker --tail=3
# SIMULATED NOTIFICATION: {"order_id":<id>,"customer_name":"Validation User"}
kubectl -n quickbite exec redis-0 -- redis-cli LLEN quickbite:notifications
# 0
```

Finally, open `https://<APP_HOST>` in a browser, place an order through the storefront, and confirm the toast `Order #<id> confirmed!`.

Operations

Inspecting the Platform

```
kubectl -n quickbite get all,pvc,networkpolicy,pdb,hpa
kubectl -n quickbite get events --sort-by=.lastTimestamp | tail -n 30
kubectl -n quickbite top pods
kubectl -n quickbite logs -l app.kubernetes.io/name=orders-api --tail=50 --prefix
```

Viewing Recent Orders (Administrative)

`GET /api/orders` is blocked at the public edge. To use it, open a port-forward to the Service:

```
kubectl -n quickbite port-forward svc/orders-api 3002:3002
curl -s http://localhost:3002/api/orders
```

Releasing a New Image Version

1. Build and push the image with a new immutable tag (Step 5).
2. Update the image tag in the Deployment manifest under `kubernetes/06-apps/`, then apply it.
3. Monitor the rollout:

```
kubectl apply -f kubernetes/06-apps/orders-api-deployment.yaml
kubectl -n quickbite rollout status deployment/orders-api
```

Rolling Back

```
kubectl -n quickbite rollout history deployment/orders-api
kubectl -n quickbite rollout undo deployment/orders-api
```

After a rollback, update the manifest so it matches the running version. Otherwise the next `kubectl apply` reintroduces the faulty tag.

Applying Configuration Changes

Pods do not reload environment variables from ConfigMaps or Secrets, and Nginx does not reload its configuration. After you change them, restart the affected workloads:

```
kubectl -n quickbite rollout restart deployment/frontend
kubectl -n quickbite rollout restart deployment/restaurant-api deployment/orders-api
```

Restoring from Backup

1. Scale `restaurant-api` and `orders-api` to 0 (temporarily delete or suspend their HPAs first).
2. Run a temporary Pod labelled `app.kubernetes.io/name: postgres-backup` that mounts `postgres-backups`, and run `pg_restore -h postgres -U quickbite -d quickbite --clean --if-exists /backups/<file>`.
3. Scale the APIs back up, restore the HPAs, and run the smoke test from Step 17.

Cleanup

Delete resources in this order so that AWS load balancers and EBS volumes are released before the VPC is removed:

```
kubectl delete -f kubernetes/07-gateway/             # releases the NLB
kubectl delete namespace quickbite                   # deletes PVCs and their EBS volumes
(reclaimPolicy Delete)
helm uninstall eg -n envoy-gateway-system
helm uninstall aws-load-balancer-controller -n kube-system
eksctl delete cluster -f infrastructure/eks/cluster.yaml --disable-nodegroup-eviction

for repo in frontend restaurant-api orders-api notification-worker; do
  aws ecr delete-repository --repository-name "quickbite/${repo}" --force --region
"$AWS_REGION"
done
aws iam delete-policy \
  --policy-arn "arn:aws:iam::${AWS_ACCOUNT_ID}:policy/AWSLoadBalancerControllerIAMPolicy"
```

Then confirm in the EC2 console that no load balancers or `available` EBS volumes tagged for the cluster remain. If the StorageClass uses `reclaimPolicy: Retain`, delete the retained volumes manually.

Out of Scope

The following are intentionally excluded from Part 1 and are delivered in Part 2:

- Container image build and publication pipelines (GitHub Actions or similar)
- Argo CD or any other GitOps controller
- Automated promotion between environments

Part 1 manifests should still be written so that automation can adopt them later. Keep one resource kind per file, avoid environment-specific values where possible, and keep all configuration in source control except Secrets.